

Johnson Algorithm

Johnson's Algorithm finds the shortest paths between every pair of vertices by combining Bellman-Ford and Dijkstra's algorithms.

Algorithm:

1. Add a new vertex S to the graph.
 - Add an edge of weight 0 from S to every vertex.
2. Run Bellman-Ford from S .
 - Calculate a value $h(v)$ for every vertex v .
 - If Bellman-Ford detects a negative-weight cycle, stop; shortest paths do not exist.
3. Reweight every edge (u, v) :
 $w'(u,v)=w(u,v)+h(u)-h(v)$
This makes all edge weights non-negative.
4. Remove the new vertex S and its edges.
5. Run Dijkstra's algorithm from every vertex.
 - For each source vertex u , calculate the shortest distance to all other vertices using the reweighted edges.
6. Convert the distances back to the original weights:
 $d(u,v)=d'(u,v)-h(u)+h(v)$
7. The resulting distance matrix contains the shortest path between every pair of vertices.

Implementation:

Start

```
void johnson_algo(vector<vector<int>> &adj){  
  
}
```

adj -> adjacency matrix

1. Add a new vertex S to the graph.
 - Add an edge of weight 0 from S to every vertex.

```
void johnson_algo(vector<vector<int>> &adj){  
    int n = adj.size()+1;
```

```

vector<vector<int>> tmp_mat(n,vector<int>(n));
for(int i=0; i<adj.size(); i++){
    for(int j=0; j<adj.size(); j++){
        tmp_mat[i][j] = adj[i][j];
    }
}
for(int i=0; i<n; i++){
    tmp_mat[i][n-1] = inf;
    tmp_mat[n-1][i] = 0;
}
}

```

2. Run Bellman-Ford from S.

- Calculate a value $h(v)$ for every vertex v .
- If Bellman-Ford detects a negative-weight cycle, stop; shortest paths do not exist.

```

void johnson_algo(vector<vector<int>> &adj){
    int n = adj.size()+1;
    vector<vector<int>> tmp_mat(n,vector<int>(n));
    for(int i=0; i<adj.size(); i++){
        for(int j=0; j<adj.size(); j++){
            tmp_mat[i][j] = adj[i][j];
        }
    }
    for(int i=0; i<n; i++){
        tmp_mat[i][n-1] = inf;
        tmp_mat[n-1][i] = 0;
    }
    vector<vector<pair<int,int>>> tmp_list(n);
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            tmp_list[i].push_back({j,tmp_mat[i][j]});
        }
    }
    bool cycle;
    vector<int> h = bellman(tmp_list,n-1,cycle);
    if(cycle){
        cout<<"Shortest path doesn't exist\n\n";
        return;
    }
}

```

}

3. Reweight every edge (u, v):

$$w'(u,v)=w(u,v)+h(u)-h(v)$$

This makes all edge weights non-negative.

4. Remove the new vertex S and its edges.

```
void johnson_algo(vector<vector<int>> &adj){
    int n = adj.size()+1;
    vector<vector<int>> tmp_mat(n,vector<int>(n));
    for(int i=0; i<adj.size(); i++){
        for(int j=0; j<adj.size(); j++){
            tmp_mat[i][j] = adj[i][j];
        }
    }
    for(int i=0; i<n; i++){
        tmp_mat[i][n-1] = inf;
        tmp_mat[n-1][i] = 0;
    }
    vector<vector<pair<int,int>>> tmp_list(n);
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if(tmp_mat[i][j] != inf){
                tmp_list[i].push_back({j,tmp_mat[i][j]});
            }
        }
    }
    bool cycle = false;
    vector<int> h = bellman(tmp_list,n-1,cycle);
    if(cycle){
        cout<<"Shortest path doesn't exist\n\n";
        return;
    }
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            tmp_mat[i][j] = tmp_mat[i][j]+h[i]-h[j];
        }
    }
    for(int i=0; i<adj.size(); i++){
        for(int j=0; j<adj.size(); j++){
            adj[i][j] = tmp_mat[i][j];
        }
    }
}
```

}

5. Run Dijkstra's algorithm from every vertex.

- For each source vertex u , calculate the shortest distance to all other vertices using the reweighted edges.

```
void johnson_algo(vector<vector<int>> &adj){
    int n = adj.size()+1;
    vector<vector<int>> tmp_mat(n,vector<int>(n));
    for(int i=0; i<adj.size(); i++){
        for(int j=0; j<adj.size(); j++){
            tmp_mat[i][j] = adj[i][j];
        }
    }
    for(int i=0; i<n; i++){
        tmp_mat[i][n-1] = inf;
        tmp_mat[n-1][i] = 0;
    }
    vector<vector<pair<int,int>>> tmp_list(n);
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if(tmp_mat[i][j] != inf){
                tmp_list[i].push_back({j,tmp_mat[i][j]});
            }
        }
    }
    bool cycle = false;
    vector<int> h = bellman(tmp_list,n-1,cycle);
    if(cycle){
        cout<<"Shortest path doesn't exist\n\n";
        return;
    }
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if(tmp_mat[i][j] != inf){
                tmp_mat[i][j] = tmp_mat[i][j] + h[i] - h[j];
            }
        }
    }
    for(int i=0; i<adj.size(); i++){
        for(int j=0; j<adj.size(); j++){
            adj[i][j] = tmp_mat[i][j];
        }
    }
    vector<vector<pair<int,int>>> tmp_list2(adj.size());
    for(int i=0; i<adj.size(); i++){
        for(int j=0; j<adj.size(); j++){
            if(adj[i][j] != inf){
```

```

        tmp_list2[i].push_back({j,adj[i][j]});
    }
}
for(int i=0; i<adj.size(); i++){
    vector<int> dist = dijkstra(tmp_list2,i);
    for(int j=0; j<adj.size(); j++){
        adj[i][j] = dist[j];
    }
}
}

```

6. Convert the distances back to the original weights:

$$d(u,v)=d'(u,v)-h(u)+h(v)$$

```

void johnson_algo(vector<vector<int>> &adj){
    int n = adj.size()+1;
    vector<vector<int>> tmp_mat(n,vector<int>(n));
    for(int i=0; i<adj.size(); i++){
        for(int j=0; j<adj.size(); j++){
            tmp_mat[i][j] = adj[i][j];
        }
    }
    for(int i=0; i<n; i++){
        tmp_mat[i][n-1] = inf;
        tmp_mat[n-1][i] = 0;
    }
    vector<vector<pair<int,int>>> tmp_list(n);
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if(tmp_mat[i][j] != inf){
                tmp_list[i].push_back({j,tmp_mat[i][j]});
            }
        }
    }
    bool cycle = false;
    vector<int> h = bellman(tmp_list,n-1,cycle);
    if(cycle){
        cout<<"Shortest path doesn't exist\n\n";
        return;
    }
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if(tmp_mat[i][j] != inf){

```

```

        tmp_mat[i][j] = tmp_mat[i][j] + h[i] - h[j];
    }
}
for(int i=0; i<adj.size(); i++){
    for(int j=0; j<adj.size(); j++){
        adj[i][j] = tmp_mat[i][j];
    }
}
vector<vector<pair<int,int>>> tmp_list2(adj.size());
for(int i=0; i<adj.size(); i++){
    for(int j=0; j<adj.size(); j++){
        if(adj[i][j] != inf){
            tmp_list2[i].push_back({j,adj[i][j]});
        }
    }
}
for(int i=0; i<adj.size(); i++){
    vector<int> dist = dijkstra(tmp_list2,i);
    for(int j=0; j<adj.size(); j++){
        adj[i][j] = dist[j];
    }
}
for(int i=0; i<adj.size(); i++){
    for(int j=0; j<adj.size(); j++){
        if(adj[i][j] != inf){
            adj[i][j] = adj[i][j] - h[i] + h[j];
        }
    }
}
cout<<" ";
for(int i=0; i<adj.size(); i++){
    cout<<" "<<i<<" ";
}
cout<<"\n";
for(int i=0; i<adj.size(); i++){
    cout<<i<<" ";
    for(int j=0; j<adj.size(); j++){
        if(adj[i][j]==inf){
            cout<<"inf ";
        }else{
            cout<<" "<<adj[i][j]<<" ";
        }
    }
}

```

```

    }
}
cout<<"\n";
}
}

```

Full Code:

```

#include<iostream>
#include<vector>
#include<queue>
using namespace std;
const int inf = 1e8;

vector<int> bellman(vector<vector<pair<int,int>>> adj,int s,bool
&cycle){
    int v = adj.size();
    vector<int> dist(v,inf);
    dist[s] = 0;

    for(int k=0; k<v-1; k++){
        for(int i=0; i<adj.size(); i++){
            for(int j=0; j<adj[i].size(); j++){
                int s = i;
                int d = adj[i][j].first;
                int w = adj[i][j].second;

                if(dist[s] != inf){
                    int newD = dist[s] + w;
                    if(newD < dist[d]){
                        dist[d] = newD;
                    }
                }
            }
        }
    }

    bool negCycle = false;
    for(int i=0; i<adj.size(); i++){
        for(int j=0; j<adj[i].size(); j++){

```

```

        int s = i;
        int d = adj[i][j].first;
        int w = adj[i][j].second;
        if(dist[s]!=inf && dist[s]+w<dist[d]){
            negCycle = true;
            break;
        }
    }
    if(negCycle){
        break;
    }
}
cycle = negCycle;
return dist;
}

vector<int> dijkstra(vector<vector<pair<int,int>>> adj,int s){
    int v = adj.size();
    vector<int> dist(v,inf);
    dist[s] = 0;
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<
int,int>>> visited;
    visited.push({0,s});
    while(!visited.empty()){
        int u = visited.top().second;
        int d = visited.top().first;
        visited.pop();
        for(int i=0; i<adj[u].size(); i++){
            int ver = adj[u][i].first;
            int dis = adj[u][i].second;
            int newD = dis + d;
            if(newD < dist[ver]){
                dist[ver] = newD;
                visited.push({newD,ver});
            }
        }
    }
    return dist;
}

void johnson_algo(vector<vector<int>> &adj){
    int n = adj.size()+1;

```



```

vector<vector<int>> tmp_mat(n,vector<int>(n));
for(int i=0; i<adj.size(); i++){
    for(int j=0; j<adj.size(); j++){
        tmp_mat[i][j] = adj[i][j];
    }
}
for(int i=0; i<n; i++){
    tmp_mat[i][n-1] = inf;
    tmp_mat[n-1][i] = 0;
}
vector<vector<pair<int,int>>> tmp_list(n);
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++){
        if(tmp_mat[i][j] != inf){
            tmp_list[i].push_back({j,tmp_mat[i][j]});
        }
    }
}
bool cycle = false;
vector<int> h = bellman(tmp_list,n-1,cycle);
if(cycle){
    cout<<"Shortest path doesn't exist\n\n";
    return;
}
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++){
        if(tmp_mat[i][j] != inf){
            tmp_mat[i][j] = tmp_mat[i][j] + h[i] - h[j];
        }
    }
}
for(int i=0; i<adj.size(); i++){
    for(int j=0; j<adj.size(); j++){
        adj[i][j] = tmp_mat[i][j];
    }
}
vector<vector<pair<int,int>>> tmp_list2(adj.size());
for(int i=0; i<adj.size(); i++){
    for(int j=0; j<adj.size(); j++){
        if(adj[i][j] != inf){
            tmp_list2[i].push_back({j,adj[i][j]});
        }
    }
}

```

```

    }
}
for(int i=0; i<adj.size(); i++){
    vector<int> dist = dijkstra(tmp_list2,i);
    for(int j=0; j<adj.size(); j++){
        adj[i][j] = dist[j];
    }
}
for(int i=0; i<adj.size(); i++){
    for(int j=0; j<adj.size(); j++){
        if(adj[i][j] != inf){
            adj[i][j] = adj[i][j] - h[i] + h[j];
        }
    }
}
cout<<" ";
for(int i=0; i<adj.size(); i++){
    cout<<" "<<i<<" ";
}
cout<<"\n";
for(int i=0; i<adj.size(); i++){
    cout<<i<<" ";
    for(int j=0; j<adj.size(); j++){
        if(adj[i][j]==inf){
            cout<<"inf ";
        }else{
            cout<<" "<<adj[i][j]<<" ";
        }
    }
    cout<<"\n";
}
}

int main(){
    int v,e;
    cin>>v>>e;
    vector<vector<int>> adj(v,vector<int>(v));
    for(int i=0; i<v; i++){
        for(int j=0; j<v; j++){
            if(i==j){
                adj[i][j] = 0;
            }else{

```

```

        adj[i][j] = inf;
    }
}
for(int i=0;i<e; i++){
    int s,d,w;
    cin>>s>>d>>w;
    adj[s][d] = w;
}
johnson_algo(adj);
return 0;
}

```

```

4 5
0 1 -5
0 2 2
0 3 3
1 2 4
2 3 1
    0    1    2    3
0  0   -5   -1   0
1 inf  0    4    5
2 inf inf  0    1
3 inf inf inf  0

```